

C语言与程序设计

The C Programming Language

第2章 C语言的基本元素

王多强

1097412466

dqwang@hust.edu.cn

华中科技大学计算机学院

学习一门语言， 要关注两方面的问题： 语法和语义

语法：指语言的文法结构、组词造句的规则

——不能写错别字、病句

——编译器一般只能检查语法错误

语义：指字词、符号、句子的含义， 要能正确表达逻辑含义

——意思表达准确

——语义错误一般只能由程序员来发现和纠正。

—— 本章学习**C语言的基本要素**， 奠定编程的**正确语法规义基础**



第二章第一部分：

2.1 C语言的词法元素

2.2 标识符和关键字

2.3 数据和数据类型

2.4 常量与变量

2.1 C语言的词法元素

C语言源程序本质上是一个**字符序列**，由各种**字符**组成。

编程就是用C语言规定的合法规则“写”一篇“作文”。

```
#include <stdio.h>
void show(char str[]);
int main(void)
{
    char name[20];
    printf("Input your name please!\n");
    gets(name);
    printf("Hello %s!\n",name);
    show(name);
    return 0;
}
```

涉及：字符、组词、造句、段落、组织结构等多个方面内容。

2.1.1 C语言的字符集

在屏幕上可显现、人眼可看到其图形式样的符号

C语言的字符集为**ASCII字符集**中的所有**可打印字符**及少量控制字符，包括：

◆ **英文字母**：a~z 和A~Z

◆ **数字字符**：0~9

◆ **特殊字符**：! " # % & ' () * + , - . / : ; < >

= ? [] \ ^ _ { } | ~

29个英文标点符号

◆ **空白字符**：空格、换行符、制表符等

中文能不能用？

能写在程序里，但只能作为字符数据，用作字符常量或字符串的文字内容以及注释中，其它任何地方不能用中文。

三字符序列（自学，自己查找资料了解）：

以**两个连续问号**开头的三字符序列(教材表2-1, p21)。

所有的三字符序列都要用相应的单个字符替换，这种替换发生在其他任何处理之前。

例如，

```
int a??(4??)=??<0??>;
```

被替换成

```
int a[4]={0};
```

2.1.2 词法元素

C源程序是一个**字符序列**，编译器在编译时，首先要识别出其中的**词**、**标点符号**等语法成分，即对C源程序中的字符序列进行“**分词**”操作，形成一个个**具有语义成分的基本单元**（单词、运算符、标点符号等）。

—— 这些基本单元称为**记号(Token)**。

Token是程序的基本组成单位，称为**词法元素**。

- ◆ 类似于人对一篇文章，先识别出其中的单词、基本符号。

基本方法：找出被**空格/分隔符**分割出来的成份，但不尽然。

例2.1 **sum=x+y** 表达式 (短语)

分解成 **sum**、**=**、**x**、**+**、**y** 5个记号。

其中, **sum**、**x**、**y** 是变量名字 (标识符)

=、**+** 是运算符

例2.2 **int a, b=10;** 语句 (一句话)

分解成 **int**、**a**、**,**、**b**、**=**、**10**、**;** 7个记号

其中, **int** 是类型名 (关键字)

a、**b** 是变量名 (标识符)

= 是运算符

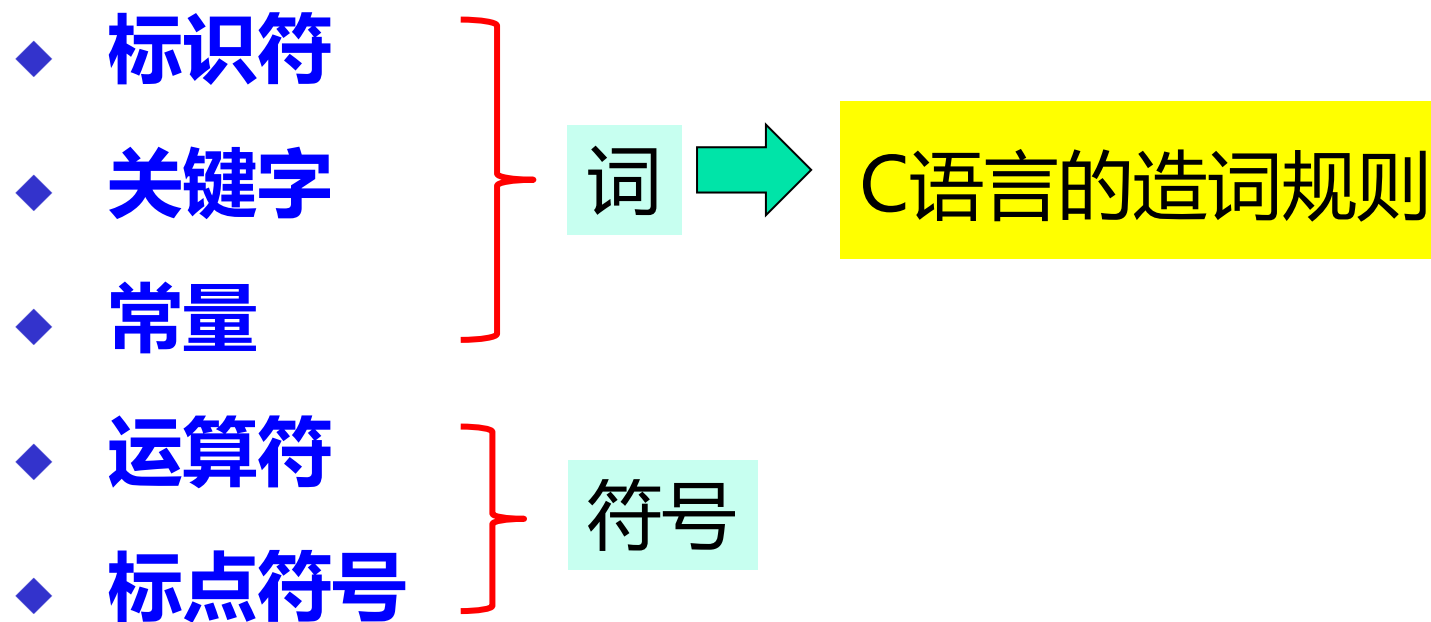
10 是数字常量

; 是标点符号

例2.3 **x+++++y**

分解成 **x**、**++**、**+**、**++**、**y** 5个记号

C语言中记号共分5类:



2.2 词的构造：标识符、关键字

2.2.1 标识符

- ◆ C语言中所有的**名字**都称为**标识符**。如：
 - 数据类型名：int、char
 - 函数名：printf、main、gcd
 - 变量名：x、y、sum
 - 符号常量名等
- ◆ **标识符是C语言中的单词**，是由一个或多个字符组成的字符序列



标识符的命名规则：

以**字母**或**下划线**开头，后跟**字母**、**下划线**或**数字**的组合。

这里，字母是指：**a~z, A~Z**，区分大小写（大小写敏感）

数字是指：**0~9**

下划线是：**_**

例：以下哪些是合法的标识符，哪些是不合法的标识符：

K



k123



_id



_id



month



time1



20_sum



_20_sum



_



not#me



_not#me



9



C语言是**大小写敏感**的，即区分字母的大小写。如：

Name、name、nAme

将被视为三个不同的标识符。

◆ 友情提示：

良好的编程风格是选择**有助于记忆且有一定含义**的标识符，
这样**可增强程序的可读性和可维护性**。

被大家认可的命名法有：**匈牙利命名法、驼峰式命名法、帕斯卡命名法**等。（自己查找资料学习和实践）

例：**匈牙利命名法**是一种编程时的命名规范。基本原则是：

变量名 = 属性 + 类型 + 对象描述

其中对象的名称要有明确含义，可以取对象名字全称或名字的一部分。要基于容易记忆、容易理解的原则。保证名字的连贯性是非常重要的。如：int iMyAge; float fManHeight

- **驼峰式命名法**：第一个单词首字母小写，后面其他单词首字母大写。 如：int myAge;
- **帕斯卡命名法**：每个单词的第一个字母都大写。 如：int MyAge;

2.2.2 关键字

关键字：是被系统预先赋予了特定含义和专门用途的标识符，
是系统的“专有名词”，又称为保留字或预定义标识符。

C90标准里定义了32个关键字

auto	break	case	char	const	continue	default	do
double	else	enum	extern	float	for	goto	if
int	long	register	return	short	signed	sizeof	static
struct	switch	typedef	union	unsigned	void	volatile	while

C99标准新增5个关键字

<code>inline</code>	<code>restrict</code>	<code>_Bool</code>	<code>_Complex</code>	<code>_Imaginary</code>
---------------------	-----------------------	--------------------	-----------------------	-------------------------

C11标准新增7个关键字

<code>_Alignas</code>	<code>_Alignof</code>	<code>_Atomic</code>	<code>_Static_assert</code>	<code>_Noreturn</code>	<code>_Thread_local</code>	<code>_Generic</code>
-----------------------	-----------------------	----------------------	-----------------------------	------------------------	----------------------------	-----------------------

详见教材表2-1 (p23)

C程序里除关键字外的其它标识符称为**用户自定义标识符**。

用户自定义的标识符不要和关键字重名。

此外，C语言还有一些**保留标识符**（C语言自己约定俗成使用的名字），如 `printf`、`scanf` 等，用户自定义标识符时也不要和这些保留标识符重名。

2.2.3 分隔符

- ◆ 不同的标识符之间要分开，不能连写。

如，`int x, y;` 不要写成 `intx,y;` 或 `intxy;`；否则无法区分。

- ◆ 最常用的分隔符：**空格**

- 其它还有：**制表符、换行符、注释符**等。

- ◆ 分隔符在语法上起**分隔单词**的作用

- **分隔符不是必须的**，但当两个相邻的单词之间如果不用分隔符就不能区分开时就必须加分隔符。

如：`int x; sum=x+y;`

- 多加分隔符对程序没影响，编译器会忽略掉多余的分隔符，

如 `sum = x + y` 等价于 `sum = x + y`

2.3 数据和数据类型

数学中，数有类型之分，如整数、实数、复数等。

C语言中要求：对所有的量（变量和常量）必须指定类型：

- ◆ **数值型**数据：整型、浮点型
- ◆ **字符型**数据：字符型、字符串型等
- ◆ **其他数据**：如图像、音频，这些数据最终都转换成数值型数据输入到计算机系统中进行处理。

数值型和字符型是计算机能够处理的基本数据类型

数据类型是一种语言的数据表达能力，类型越丰富，对客观世界的描述能力越强大。C语言提供了丰富的数据类型。

2.3.1 基本数据类型

C语言的基本数据类型只有三种：

- ◆ 表示整数的类型：称为**整型**
- ◆ 表示实数的类型：称为**浮点类型**
- ◆ 表示字符的类型：称为**字符型**

预备知识1： 指定一个量的数据类型，具体明确了什么？

(1) 存储该量所需的字节数

C语言规定了每种类型的数据存储时所需的字节数，**指定一个量的数据类型后，就确定了该量在内存中存储所占用的字节数。**

不同类型的数据所占字节数不一，如 char型是1字节，int型一般是2字节或4字节等。

一种数据类型的（一个）数据在内存中存储时所需的字节数称为该数据类型的**数据长度**（或**类型长度**）。

(2) 该量的值的取值范围。

数据长度越大（用的字节数越多，位数就越多），可表示的数据范围就越大。

如，1字节整数只能表示 $-128 \sim 127$ ，而2字节的整数可表示 $-32768 \sim 32767$ 。

所以，存储字节数确定了，量的值的取值范围也就确定了。

(3) 该量在计算机内部的表示形式

如，符号整数采用**补码**表示，浮点数采用**IEEE754**规则表示。

(4) 该量可以参加的运算

不同类型的数据可参加的运算会有所不同。如整数参加整型数据可以参与的运算，浮点数参与浮点数类型可以参加的运算。

预备知识2：数据类型名和类型修饰符

- ◆ C语言给每种类型都命了一个**类型名**
 - **类型名是关键字**，通常取自该类型的英文名或“简称”。
- ◆ **类型修饰符**：在说明数据类型的时候，可以用**类型修饰符**从一个基本类型**派生**出一个**扩展类型**。 **派生类型**

常用的类型修饰符有：

- ◆ 表示**有符号数据**：**signed**
- ◆ 表示**无符号数据**：**unsigned**
- ◆ 表示“短”整型数：**short**
- ◆ 表示“长”整型数：**long**

具体用法及含义见后

C语言的基本数据类型

1) 字符型

基本类型名：**char**

存储长度：**1字节(8bit)**

表示形式：有两种，

(I) 用一对英文**单引号**括起来**单个字符**

如：**'a'**、**'1'**、**'@'**

注：字符数据指**单引号里面的字符**，单引号仅是字符数据的**“界定符”**，不是字符数据本身。

(II) 转义字符序列

■ 字符数据的值：

字符的图形（如 'A'）仅是“人眼看起来的模样”。

在计算机内部，字符数据用**ASCII码**表示——字符数据的值等于字符的**ASCII码值**（一个1字节的“**小整数**”，值：**0~127**）。

◆ **字符可以参与整数运算**，把其**ASCII码值**当作“**小整数**”使用：

如：**5+'a'** 等价于 **5+97**

■ 字符数据可分为：

有符号字符：**signed char** (或 **char**)，将其字节内的最高位作为符号位，其余位为数值位，取值范围**-128 ~127**。

无符号字符：**unsigned char**，将其字节内的所有位都作为数值位，取值范围**0~255**。

2) 整型

整型用于表示**有符号或无符号整数**，有以下**基本形**和**扩展形**：

(1) **基本形**：**int**，也称为**标准int型**，为**有符号整数**

存储长度：一般等于系统的**机器字长**，16位系统中int型长度为**2字节**，32位系统中int型长度为**4字节**。

数据范围：2字节（16bit）整数：-32768~32767

4字节（32bit）整数： $-2^{32-1} \sim 2^{32-1}-1$

注：int型是**有符号整数**，全称：signed int，signed可以缺省，简称为int。

机器字长：计算机系统中数据通路的宽度，16位机的机器字长为16位，32/64位机的机器字长为32/64位。

(2) 扩展形:

◆ 长整型、短整型:

short int: “短整型”, 长度一般为2个字节 (不长于int型)。

可简称为**short**, 如 short int x; 或 short x;

long int: “长整型”, 理论长度不短于int型 (通常为1-2倍),

16位机long int型为4字节

32位机long int型为4字节

可简称为**long**, 如 long int y; 或 long x;

- ◆ **有符号整型**：带有符号位，值有正负之分

int、short、long int 默认都是有符号数。

注：全称分别是**signed int**、**signed short int**、**signed long int**，其中修饰符signed可以省略，简称为 int、short、long int。

- ◆ **无符号整型**：所有位都是数值位，值均为非负数

unsigned int：无符号整型

unsigned short int：无符号短整型

unsigned long int：无符号长整型

综上，C语言中整型数据类型有

- ◆ **int**: (signed int) 标准整型
- ◆ **short int**: (signed short int, 或 short) 短整型
- ◆ **long int**: (signed long int, 或 long) 长整型
- ◆ **unsigned int**: (unsigned) 无符号整型
- ◆ **unsigned short int**: (unsigned short) 无符号短整型
- ◆ **unsigned long int**: (unsigned long) 无符号长整型
- ◆ 另外，还有 **long long int**: 比long int更长的整型

有符号数

(3) 整型数据的取值范围

(1) 机器字长为2B (2Byte, 16位) 时, 相应取值范围为:

- **int** : -32768~32767,  短型整数与int型相同
- **unsigned int** : 0 ~ 65535
- **long int** : -2147483648 ~ 2147483647
- **unsigned long int** : 0 ~ 4294967295

(2) 字长机器为4B (4Byte, 32位) 时, 相应取值范围为:

- **short int** : -32768~32767
- **unsigned short int** : 0 ~ 65535
- **int** : -2147483648 ~ 2147483647  长型整数与int型相同
- **unsigned int** : 0 ~ 4294967295

◆ 短整型数据有什么好处？

- 节约存储空间，内存中只占2个字节（包括在数据文件中）。
- 但CPU处理时，会把short型数据扩展成int型处理，所以short型数据的处理效率和int型是一样的。

◆ 长整型的处理效率不高于int型，慎用。

◆ 输出不同整型数据的**格式符**

- **%d**: 输出标准int, 如 int x=5; printf("**%d**", x);
- **%hd**: 输出short int, 如 short s=5; printf("**%hd**", s);
- **%ld**: 输出long int, 如 long l=5; printf("**%ld**", l);
- **%u**: 输出无符号int, 如 unsigned u=5; printf("**%u**", u);
- **%c**: 输出char, 如 char c='a'; printf("**%c**", c);

3) 浮点类型

(1) C语言的浮点类型有以下3种:

- ◆ **float**: 单精度浮点类型, 4字节长度
- ◆ **double**: 双精度浮点类型, 8字节长度
- ◆ **long double**: (c99新增类型, 自学)

(2) 浮点数的机器表示

在计算机中表示浮点数要解决以下三个问题:

- ◆ **正负号**
- ◆ **尾数**
- ◆ **阶码**

这是一个世界级的问题!
曾有人因此获得了图灵奖

浮点数的表示 (IEEE754标准) :

一个浮点数V可表示为以下形式 (类似科学计数法) :

$$V = (-1)^s \times M \times 2^E$$

↓↓↓
数符 **尾数** **指数**



Prof. William Kahan

发明者

1989
ACM Turing
Award Winner!

编码规则: IEEE754标准 (IEEE二进制浮点数算术标准), 规定了单、双精度浮点数的编码规则

数符: 用1位表示, 表示数的正、负, **0表示正数、1表示负数;**

尾数: 采用**规格化形式**: 1.XXXXX, 但只存储XXXXX部分, 1省略,
用23位 (单精度) 或52位 (双精度) **原码表示;**

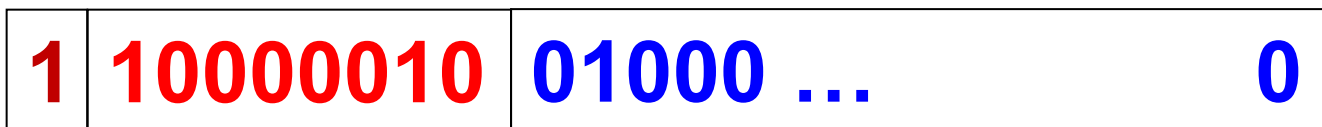
指数: 代表2的幂指数, 用8位 (单精度) 或11位 (双精度) **移码表示。**

■ 如: $(-10.0)_{10} = (-1010.0)_2 = (-1.01 * 2^3)_2 = ((-1)^1 * 1.01 * 2^3)_2$

非规格化尾数

规格化尾数

标准形式



符号位: 1位

尾数: 用**原码**表示, 23位 (单精度; 双精度是52位)

阶码: 用**移码**表示, 8位 (单精度) 或11位 (双精度)

阶码 = 偏置常数 + 指数

偏置常数: 单精度127 或 双精度1023

《计算机系统基础》课程里再做详解

- 尾数的位数决定值的**精度**，指数的位数决定值的**范围**

- **float**: **4字节**，其中符号1b，指数8b，尾数23b

其精度大约为**7位十进制数**

范围约 **$10^{-38} \sim 10^{+38}$** 。

- **double**: **8字节**，其中符号1b，指数11b，尾数52b

其精度大约为**15位十进制数**

范围约为 **$10^{-308} \sim 10^{+308}$** 。

- **long double**: 很多编译器将long double处理为double，在某些系统中，它占用**10或12B**。很少被使用。

- 浮点数的表示本质上还是有限个0-1的组合，和整数等类型类似，能表示的数据个数是有限的，是**离散**、**不连续**、**非无限的有限个数据**，且大多数情况下**不能精确表示**。（查阅资料自学）

如， $(0.5)_{10}$ 的机内表示：

$$(0.5)_{10} = (0.1)_2 = (1.0 \times 2^{-1})_2 \longrightarrow \text{可以精确表示}$$

再如， $(0.1)_{10}$ 在计算机内能不能精确表示呢？ **不能精确表示**

$$\begin{aligned}(0.1)_{10} &= (0.00011001100110011001100\dots)_2 \\ &= (1.10011001100110011001100\dots \times 2^{-4})_2\end{aligned}$$

由于只能保存23+1位，所以实际保存的数据大小是：

$$(1.100110011001100110011001100 \times 2^{-4})_2$$

数学上不绝对等于0.1，**表示误差** $\approx 9.54 \times 10^{-8}$

带来的问题:

- 存在**表示误差**、**计算误差**、**溢出**等问题。
- 不能使用**`==`**和**`!=`**运算符直接比较两个浮点数相等或不相等。

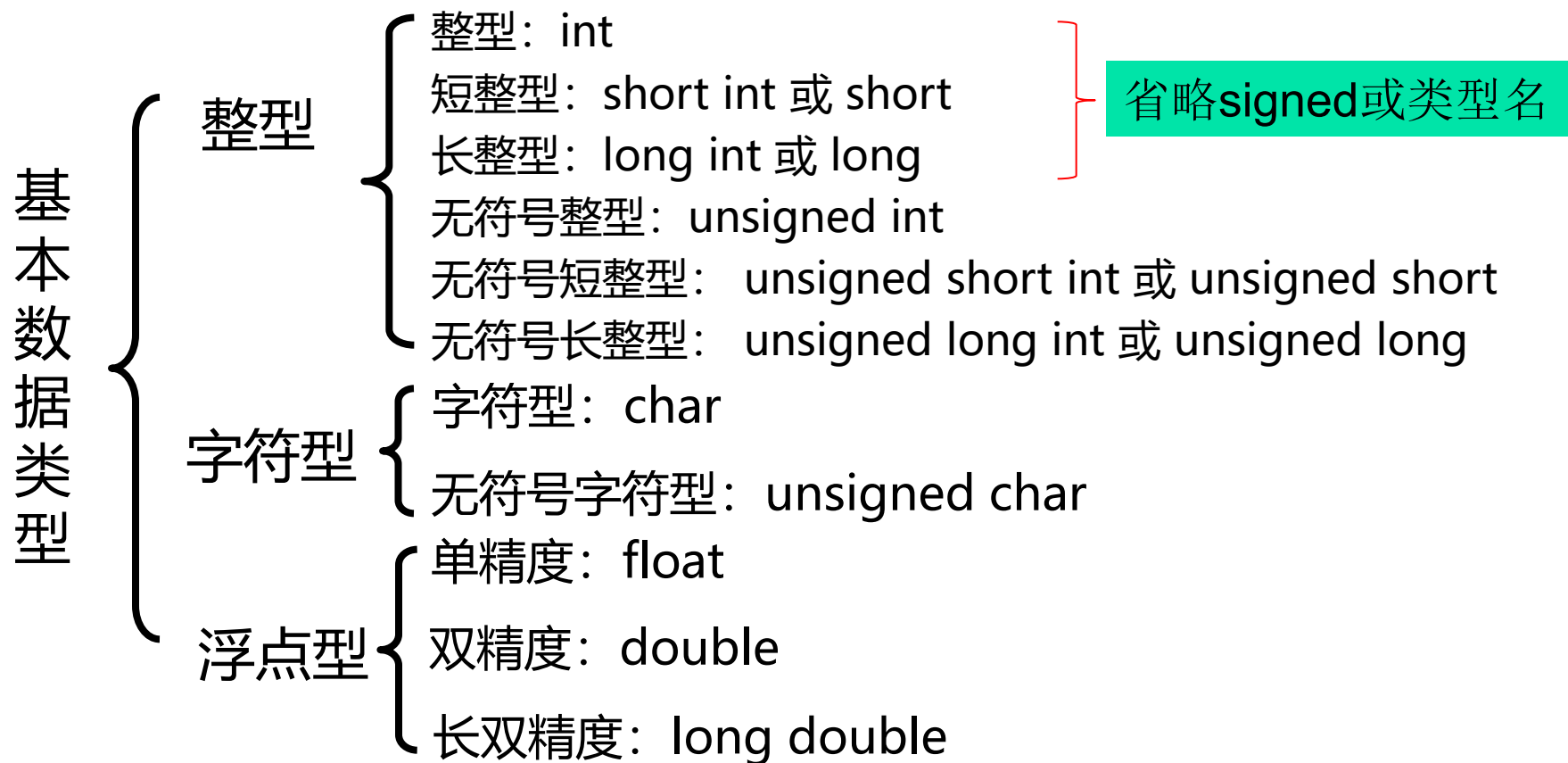
两个在数学上不相等的浮点数，可能在机内的表示相同，反之亦有可能。

解决方案：可以预定一个小正数epsilon，当两个浮点数 f_1 和 f_2 的差（绝对值）小于epsilon时，认定两个浮点数相等。

$$f_1 == f_2 \iff |f_1 - f_2| < \text{epsilon}$$

关于浮点数的问题，现在可以自行查阅资料学习，或在以后的课程里学习。

经过**基本数据类型名**和**类型修饰符**“组合”后，得到的C语言的基本数据类型有：



注：每种类型有不同的字节长度和可表示的数据范围，应根据应用的需要选择合适的类型表示数据。

2.3.2 sizeof 运算符的使用

C语言提供了一个运算符 **sizeof**，可以求出一种数据类型的大小（即类型长度——一个该类型的数据所占的字节数）。

用法：

形式1： **sizeof(数据类型)**，如 sizeof(int)、sizeof(double)

含义：自动计算指定数据类型的大小

形式2： **sizeof(变量)**，如 **float x; sizeof(x)**

含义：自动计算**变量所属数据类型**的大小

举例： printf("sizeof(int)=**%d**\n", **sizeof(int)**);

或： double **y=3.1415**;

printf("sizeof(double)=**%d**\n", **sizeof(y)**);

2.3.3 复杂数据类型

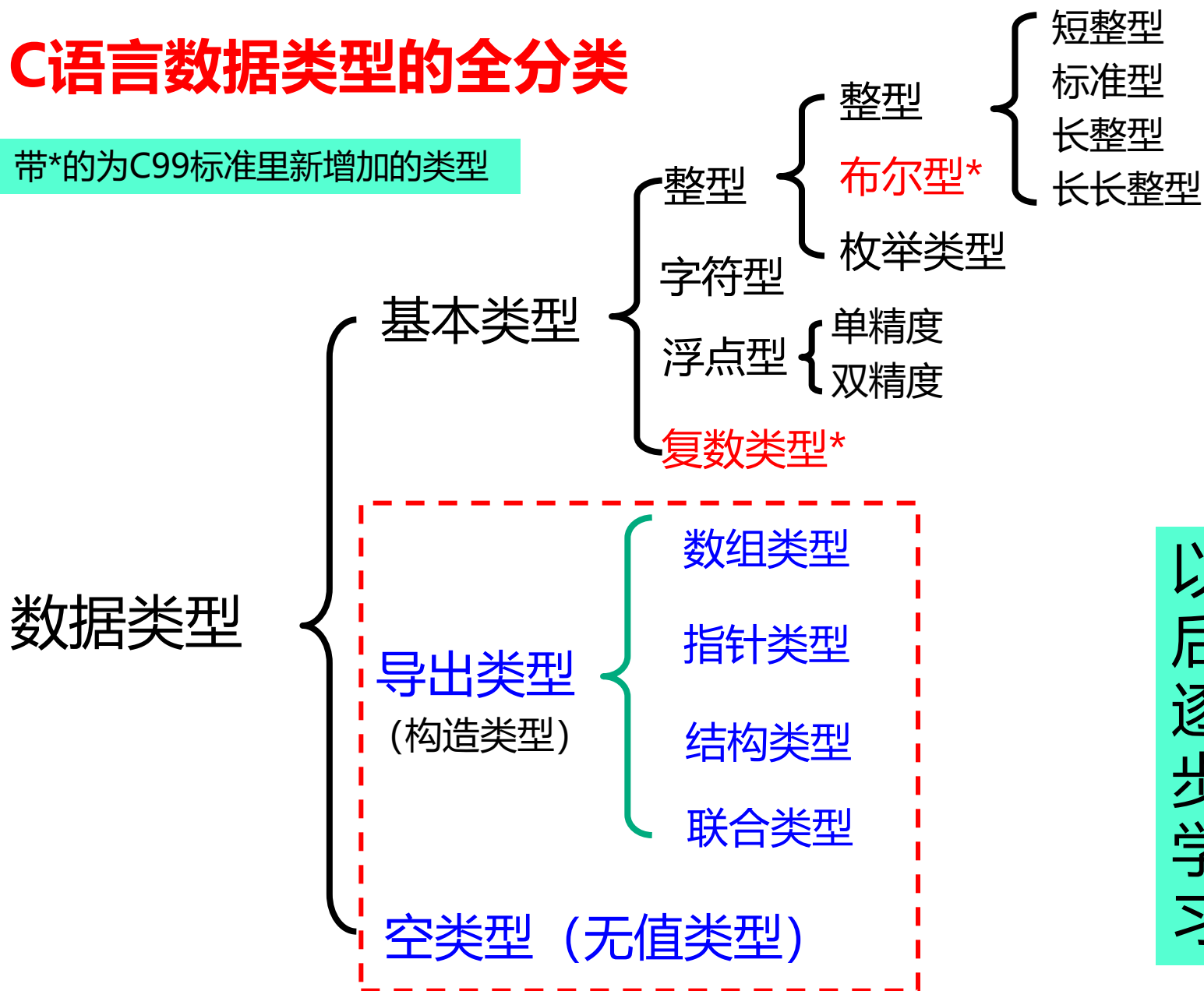
是不是想到还有下面的问题：

- ◆ **数组**是什么类型，怎么没提到？
- ◆ 前面都是基本数据类型，比较简单。有没有复杂一点的？ **复杂数据类型**长什么样？
- ◆

以后逐步学习

C语言数据类型的全分类

带*的为C99标准里新增加的类型



以后逐步学习

注意数学概念里的“数”和计算机里的“数”的异同：

相同点：数据的值有**类型**、**大小**、**正负**，**可参与各种运算**

不同点：

◆ **无限**和**有限**的区别：

- 数学里的数往往是无限的：如整数、实数
- 计算机里的数都是有限的：任何数据类型的取值范围都是有限制——其存储长度只有有限个字节（位），可表示的数的大小有限，“无限”在计算机世界里是不存在的；

◆ **连续**和**离散**的区别：

- 数学里的数一般是连续的（主要是实数）
- 计算机里的数都是离散的：编码表示、位数有限，每个位只有0或1两种变化，可表示的数的总个数是确定且有限的。数学里的绝大部分数在计算机里是无法精确表示的。

◆ 计算机中，合法的数参与运算，结果可能是非法的——溢出，结果的值超出了该数据的类型可表示的范围。

2.4 常量与变量

常量：值不变的量称为常量；

变量：值可以改变的量称为变量。

- ◆ 任何量在程序执行期间都以一定的存储空间（若干字节）、以二进制编码的方式存储在计算机内存中，统称为数据。
 - 常量所占存储单元的内容不能被改变；
 - 变量所占存储单元的内容可以被改变。
- ◆ 不管是常量还是变量，C语言要求它们都必须要有确定的类型，而数据类型决定了它们的存储形式和取值约束。

先研究：常量

C程序的常量有两种形式：文字常量和符号常量

2.4.1 文字常量

◆ 文字常量是指在源程序中直接写出来的常量，也叫字面常量；

如： `int x=12345;`

`float f = 3.14;`

`char c='a';`

`char name[20]= "Zhang Hua" ;`

这里， 12345、 3.14、 'a' 、 20、 "Zhang Hua"是文字常量，
而x、 f、 c、 name是变量。

根据数据的类型，文字常量又分为**整型常量**、**浮点型常量**、**字符常量**、**字符串常量**

1. 整型常量：在程序中代表一个整数

- ◆ 根据**进位制**的不同，源程序中的整型常量有三种表示形式，通过**前缀字符**区分：

- **十进制**： 无前缀， 如： **12345**
- **八进制**： 前缀为**0**， 如： **012345**
- **十六进制**： 前缀为**0x**或**0X**， 如： **0x12345** 或 **0X12345**

例：十进制的**31**可写成 **31**、**037**、**0x1f** 或 **0X1F**

十进制

八进制

十六进制

注：它们在计算内部的存储表示是一样的，只是源程序中字面表示不同而已。

整型常量可以加**后缀**，表示一些**扩展类型**：

后缀符号	含义	举例	备注
u或U	unsigned型整数	5u 或 5U	
l 或 L	long型整数	5l 或 5L	
ul 或 UL	unsigned long型整数	5ul 或 5UL	
ll 或 LL	long long型	5ll 或 5LL	C99以上标准支持
ull 或 ULL	unsigned long long型	5ull 或 5ULL	C99以上标准支持

注：无后缀时，一般仅表示int。

- ◆ 各种整型常量**存储所占字节数、取值范围、数据在内存存储时的编码方式**等，都和前面讲的整型数据类型相关内容一致。
- ◆ 书写常量要注意值的大小要在其类型规定的范围内。

当书写的常量值超出指定类型的范围时，如：

1000000000000000000;

其最终值的大小、类型的确定等，相关规则很复杂，标准化前的C语言、C89和C99中各不相同，自行查阅资料了解一下。

2. 浮点型常量：在程序中表示一个浮点数

- ◆ 浮点型常量有两种表示形式：

(1) **带小数点的十进制数**形式， 如：**23.7**, **14.**, **.126**

- ◆ **整数部、小数部都可以省略**，但不能都省略。

(2) **指数形式**：科学计数法

形式：**尾数e指数**

- ◆ 尾数部分的书写规则与（1）相同，且**可以没有小数点**

- ◆ 指数部分：**e(E)±n**，代表**10^{±n}**，且**n必须为整数**

如：**45e-3** 表示 45×10^{-3}

.15e5 表示 0.15×10^5

浮点数后缀：浮点型常量也可以使用后缀来指定扩展类型

- **f**或**F**：指定为**float**类型，如：**0.1f** 或 **0.1F**（4字节存储）

无后缀时默认为double型数据（8字节存储）。

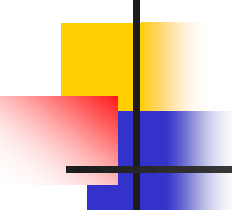
- **l**或**L**：指定为**long double**类型，如：**0.1l** 或 **0.1L**

如：**0.1** double型，8字节存储

0.1f float型，4字节存储

0.1L Long double型，更多字节存储

1F? 非法的表示

- 
-
- ◆ 各种浮点型常量存储所占字节数、取值范围、数据在内存存储时的编码方式等，也都和前面讲的浮点数据类型相关内容一致。
 - ◆ 书写浮点常量要注意值的大小要在其类型规定的范围内。

3. 字符常量：指**单个字符**数据。

- ◆ 字符常量有两种表示形式：

(1) 用**一对单引号**括起一个字符

如：'a'， '1'， '&'

- ◆ **单引号**是定义字符常量时用的**定界符**，不是数据本身
- ◆ 必须用一对**英文单引号**，中文不可以
- ◆ 只能包含一个字符： 'abc' ❌
- ◆ 不能用双引号，双引号是字符串的定义： "a" ❌

(2) 字符常量的第二种表示形式：转义字符

转义序列：是以 \ 开头的一种特殊字符序列。

\ 称为转义序列的**前导符**，表示转义序列的开始。

◆ 转义序列有两种具体表示形式：

(a) \ 仅后跟**一个**其它字符，并将**整体转义成另一字符**，如：

- '\a'：表示**响铃**（ASCII码等于7）
- '\n'：表示**换行符**（ASCII码等于10）
- '\t'：表示**水平制表符**（ASCII码等于9）

作为字符数据，依然
要用**单引号**将转义序
列括起来，而**整个转
义序列表示一个字符**

这样的转义序列称为**字符转义序列**，通常用来表示**控制字符**
或一些**特殊字符**

(b) \ 后跟1-3位的八进制数或x引导的1~2位的十六进制数

- ◆ 后跟八进制数的形式: \ooo, 其中 o 代表八进制数码
- ◆ 后跟十六进制数的形式: \xhh 或 \Xhh, 其中 h 代表十六进制数码, 数字前必须要有前导符x 或 X, 表示是十六进制。

数的大小 = 需要表示的字符的ASCII码值

—— 这样的转义序列称为**数字转义序列**, 可表示任何字符。

例如, 字符 'A' (ASCII值65) : '\101'、'\x41'

水平制表符'\t': '\11'、'\011' 或 '\x9'、'\x09'
(ASCII值9)

几个特殊的转义字符表示：

注：表示字符数据时，单引号不能少

'\n'：换行符

```
printf("Hello %s!\n",name);
```

'\\'：反斜杠

\ 被用作转义序列的前导符，故不能单独使用。

如：'\' ❌

```
putchar('\\');
```

'\''：单引号

单引号是字符数据的界定符，不能单独使用。

如：'' ❌

```
putchar('\');
```

'\"'：双引号

也可以表示成：''''

```
putchar('\"');
```

```
putchar('\"');
```

'\0'：ASCII码为0的字符

写成：'\0' 行吗？ ❌

其它转义字符请见表2-3，还有一些注意事项，也具体见教材34页

4. 字符串常量

- ◆ **字符串**是有限个字符的序列。
- ◆ 字符串常量的表示：用一对**双引号**将**0个或多个字符**括起来。

如： **"string"** 包含6个字符的字符串

"" 包含**0个**字符的字符串，称为**空字符串**

- **双引号**是定义字符串常量的**定界符**，不是字符串内容本身
- **字符串的长度**：字符串中包含的字符的个数。

如**"string"**的长度为6，**""** 的长度为0。

◆ 字符串中可以包含**转义字符**（一个转义序列只代表一个字符）

如: **"string\n"** 包含7个字符, 最后一个是转义字符 **\n**

"c:\tc" 包含4个字符, 第三个是转义字符 **\t**

"\101bc" 包含3个字符, 第一个是转义字符 **\101**

"\101\x42c" 包含3个字符, 第一个是转义字符 **\101**,
第二个是转义字符 **\x42**

几个特殊的字符串表示:

(1) `"\\` : 字符串中包含几个字符? 只有一个反斜杠

`"\"` 对不对? ❌

```
printf("%s", "\\");
```

(2) `"\'` : 字符串中包含几个字符? 只有一个单引号

`\"` 对不对? ✔️

```
printf("%s", '\');
```

(3) `""` 对吗? ❌

双引号是字符串界定符, 不能独立出现在字符串中, 必须用转义字符表示:

`\"` ✔️

```
printf("%s", "\\");
```

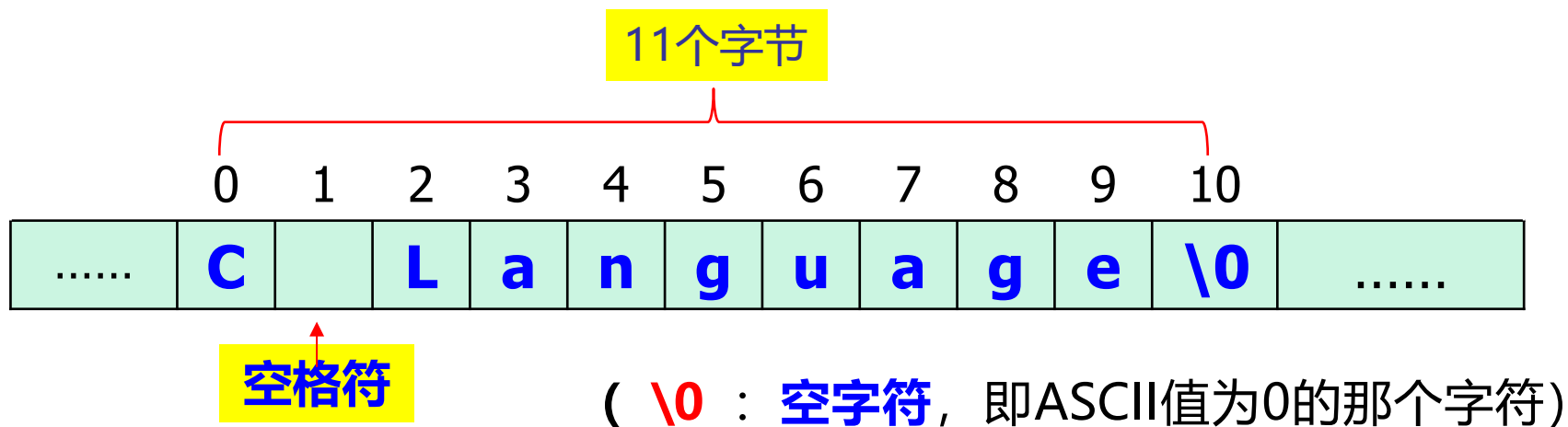
(4) `"3'40"` 和 `"3'40\"` 有何不同?

(5) `"3'40\"` 对不对? ❌

```
printf("%s", "3'40\\");
```

◆ 字符串的存储：C语言中用**连续字节空间**存储字符串常量。

如：“C Language”在内存中的存储表示为：



◆ 字符串结束标志：

C语言在**存储**字符串时，自动在串的最后加一个**空字符'\0'**作为**串**的**结束标志**。

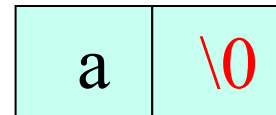
◆ 字符串的存储长度：存储字符串所用的字节数。

字符串的存储长度 = 字符串长度 + 1

思考：'a'与"a"有何区别？

'a'：字符常量，只占1B空间

"a"：字符串常量，要占2B空间



如果**字符串常量**太长，书写时一行写不下怎么办？

有两种方法：

(1) **行连接**：在前一行的末尾加**续行符**（一个 ****）再换行书写。

如：**"Hello,**

how are you";

连接后即为：**Hello, how are you**

(2) **字符串连接**：将**字符串分段**，分段后的每个子串都用双引号括起来，两个字符串之间除了换行和空格，不加其它内容。

如：**"Hello, "**

"how are you"

等效于：**Hello, how are you**



(1) printf("3'40"); 和printf("3'40\\") 各自输出什么?

3'40

3'40"

(2) printf("c:\\tc"); 在屏幕上输出什么?

c:\tc

如果是printf("c:\\tc"); 呢?

c: c

(3) printf(""); 在屏幕上输出什么?

什么都没有

2.4.2 符号常量

C程序中，可以为**文字常量**起个**名字**，这个名字称为该文字常量的**符号常量**。

编程时，原来用文字常量的地方，现在都可以用它的符号常量代替，这样可以更**简单、直观，减少出错**。

C语言中有三种定义符号常量的方法：

- (1) 用**#define**指令
- (2) 用**const**声明语句
- (3) 用**枚举类型**（在2.8节介绍）

1. 用 #define 定义符号常量

#define 是一个编译预处理指令。

用#define定义符号常量的格式为：

最后不能加额外的分号

#define 标识符 常量

如：#define PI 3.1415926

这里，PI就是符号常量，3.1415926是它所要代表的值

- ◆ **标识符**：符号常量名，必须是合法的标识符（一般习惯于大写，以和变量区分，但不是必须的）。

- ◆ **最后不要加额外的分号**

若：**#define PI 3.1415926;** 则含义大不同

例 输入圆的半径，然后计算圆的面积

```
#include <stdio.h>
```

```
#define PI 3.1415926
```

```
int main(void)
```

```
{
```

```
    int r;
```

```
    float a;
```

```
    printf("Please input radius:");
```

```
    scanf("%d",&r);
```

输入半径r的值

```
    printf("Radius = %d\nArea=%f\n" , r, PI*r*r);
```

```
    return 0;
```

```
}
```

```
Please input radius:5
Radius = 5
Area=78.539816
```

原来用文字常量的地方，
现在可以用符号常量代替

定义符号常量的好处：

- ◆ 用名字表示，常量的含义清晰，**易于理解**
- ◆ **便于编程**：用名字代替“复杂的”常量，书写简洁，修改容易

◆ **符号常量的编译预处理**（仅指#define定义的符号常量）：

在**编译预处理**阶段，预处理器会将源程序中所有符号常量的出现替换回它本来所代表的常量。

如：**PI*****r*****r** 经预处理后变成 **3.1415926*****r*****r**

—— 所以**符号常量的根本作用是便于程序员编程**，对编译器而言是不存在的（第六章讲述）。

```
#include <stdio.h>
```

```
#define PI 3.1415926
```

```
int main(void)
```

```
{
```

```
    int r;
```

```
    float a;
```

```
    printf("Please input radius:");
```

```
    scanf("%d",&r);
```

```
    printf("Radius = %d\nArea=%f\n" , r, 3.1415926 *r*r);
```

```
    return 0;
```

```
}
```

预处理后的“源程序”


```
#include <stdio.h>
```

```
#define PI 3.1415926;
```

```
int main(void)
```

```
{
```

```
    int r;
```

```
    float a;
```

```
    printf("Please input radius:");
```

```
    scanf("%d",&r);
```

```
    printf("Radius = %d\nArea=%f\n" , r, 3.1415926; *r*r);
```

```
    return 0;
```

```
}
```

多余的分号

预处理后的“源程序”

有一定编程基础的同学自学下面的程序，否则等学完循环后再回头看

例2.9 打印温度0~300华氏度和摄氏度的对照表，

温度转换公式为： $^{\circ}\text{C} = (5.0/9)(^{\circ}\text{F} - 32)$

```
#include <stdio.h>
```

```
#define LOWER 0      /* 温度的下限 */
```

```
#define UPPER 300    /* 温度的上限 */
```

```
#define STEP 20      /* 步长*/
```

```
int main(void)
```

```
{
```

```
    int fahr;
```

```
    for (fahr=LOWER; fahr<=UPPER; fahr=fahr+STEP)
```

```
        printf("%3d:%10.2f\n", fahr, (5.0/9)*(fahr-32));
```

```
    return 0;
```

```
}
```

进一步体会定义符号常量的好处：

- 符号常量使常量的含义更明确，
如：**上限、下限、步长**。
- 修改方便。

2. 用const定义符号常量

- ◆ **const**: 是关键字, **类型限定符** (**类型限定符**和**类型修饰符**类似, 放在类型之前, 表示对数据类型的一种性质扩展)。

- ◆ 用const声明常量的格式为:

const 类型名 标识符 = 常量;

分号不能少

对符号常量初始化

例如: **const double PI=3.14159;**

const int YES=1, NO=0;

用const声明常量时必须初始化。

用const和#define定义的符号常量的区别？

- **const**声明的标识符本质上是一个**只读变量**

对const声明的符号常量（**实质是个变量**），编译时系统首先会根据其类型为该量**分配存储单元**，并把指定的初始值放入其中。该值以后**不能再被更改**（只能读不能写，故视为**常量**）。

其后程序中每次引用该标识符，都对其存储单元内容做一次**只读访问**（和变量的访问是一样的，只是只能读）。

如： **const double PI=3.14159;**

double Area = 2*PI;

(续)

- **#define**定义的标识符本身没有存储单元，只是在**编译之前**由预处理程序进行简单的文本替换，它只是实际常量的一个符号象征，**本身不是实体**。

如： **#define PI** 3.14159

double Area = 2***PI**;



对编译器而言，整条语句等价于
double Area = 2* 3.14159;

2.4.3 变量

程序中值可以改变的量称为**变量**。

1. 定义变量的一般形式：

分号不能少，表示**变量定义语句**的结束。

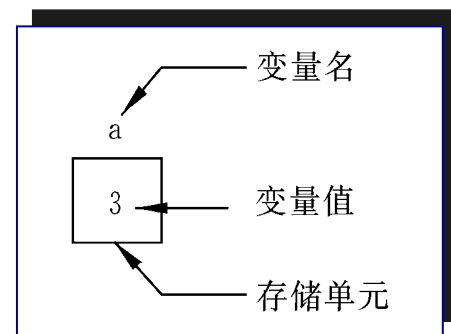
类型名 **变量名表**;

如： **int total, average;**

- **类型名**：诸如int、char等合法的数据类型名；
- **变量名**：必须是合法的标识符；
- **变量名表**：当变量名表中有多个变量时，**用逗号隔开**；
- **最后的分号不能少。**

定义变量对编译器意味着什么？

- (1) 编译器首先根据变量的数据类型，为变量在内存中分配数量等于该数据类型大小的几个连续字节作为变量存放值的空间，简称为**变量的存储空间**。
- (2) 然后**将变量的名字与这块存储空间联系起来**，作为这块内存区域的“**有名表示**”，以后程序中引用变量名就相当于引用这块存储区域。
- (3) 为变量赋值相当于将其值存储到变量对应的字节中（称为**写或存入**）。字节内容是以二进制表示的数据，称为**变量的值**。
- (4) **读**变量的值就是读取变量的字节内容。



2. 变量的初始化

在定义变量的时候为变量赋初值，称为**变量的初始化**。

如： `int count=0, sum=0;`

`char alert = '\a', c ;` /*注：这里变量c没有被初始化*/

- ◆ **=**： **赋值运算符**，表示把右边的值赋给左边的变量
- ◆ 当有多个变量并排定义且都要初始化时，每个变量必须单独初始化。

如： `int count=sum=0;` ❌

`int count=0, sum=0;` ✅

第一部分内容小结

- C语言的字符集
- 标识符的定义、关键字
- C语言的基本数据类型：字符型、整型、浮点型。
 - 字符串
 - 一些相关问题的讨论
- 常量：常量的概念，常量的类型、表示和使用
 - ◆ 文字常量：整型常量、浮点型常量、字符型常量、字符串常量
 - ◆ 符号常量：#define、const、enum类型
- 变量：概念、定义和使用

课后阅读教材2.1—2.3节